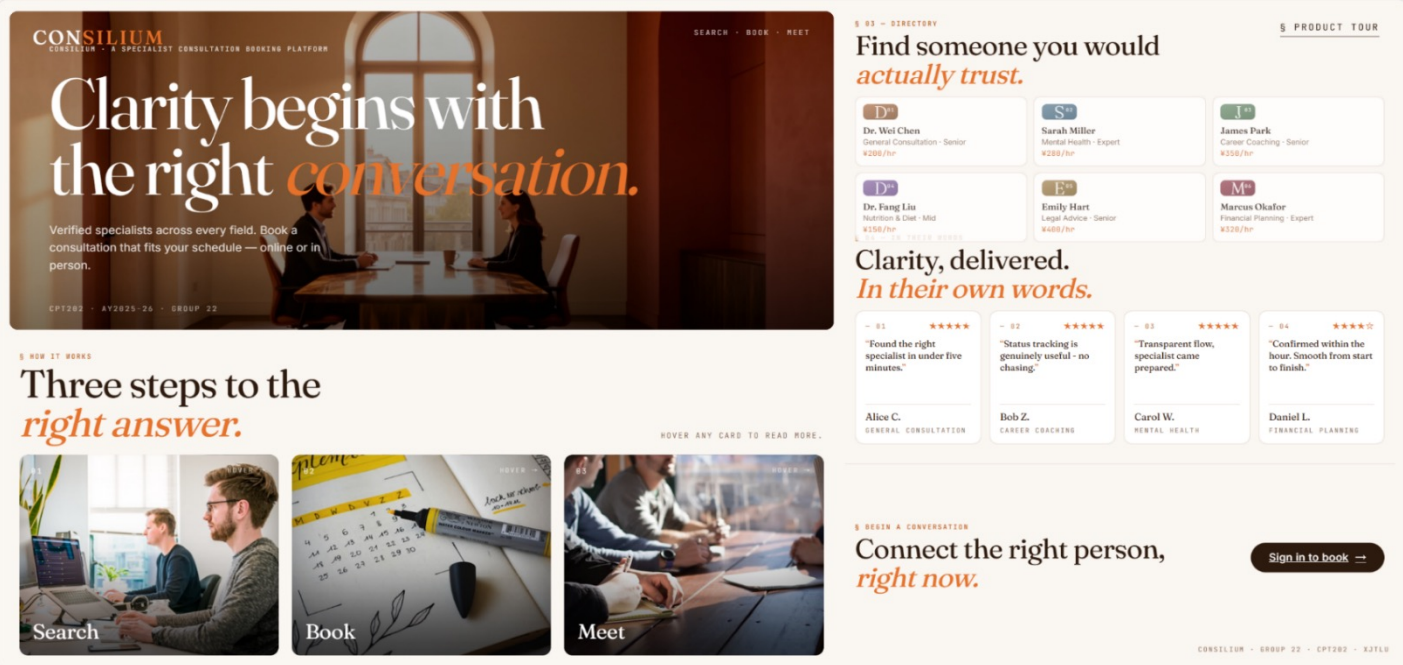
CPT202 Project - Consilium

A Specialist Consultation Booking System



System URL: http://121.41.36.107/

Group 22

1.	Jintong Zhang	2360862
2.	Kan Liu	2362244
3.	Zifan Wang	2362457
4.	Na Li	2361132
5.	Qi Cao	2360908
6.	Yuanhao Cheng	2362404
7.	Xilai Bu	2362025
8.	Shengxiang Zhu	2363335

Xi'an Jiaotong-Liverpool University
西交利物浦大學

Overview

I. Introduction & Background

II. Architectural Design

III. Software Design

IV. Software Testing

V. Conclusion

1.1 Background and Goal

User and Market:

three structural deficiencies.

- Discovery is fragmented
- Trust is opaque
- Appointment lifecycle is incomplete

Consilium:

A secure, selected and scalable solution.

- ✓ Unified Discovery
- ✓ Integrated Booking Service
- ✓ Seamless Feedbacks

“more efficient• more transparent• more manageable • more reliable”

1.2 User Requirements & Characteristics

Persona 1 — Customer

Role
Books specialist consultations

Core Needs
- Find a suitable specialist quickly
- See available time slots clearly
- Track booking status easily

Main Actions
- Search specialists
- Choose time slot
- Submit booking
- Leave review after session

Pain Points
- Unclear availability
- Too much manual communication
- Difficult to follow progress

Persona 2 — Specialist

Role
Provides consultation services

Core Needs
- Manage profile and expertise
- Control available time slots
- Handle requests efficiently

Main Actions
- Edit profile
- Set availability
- Confirm or reject bookings
- Provide meeting details

Pain Points
- Scheduling conflicts
- Too many manual requests
- Unclear session arrangements

Persona 3 — Admin

Role
Govern the platform

Core Needs
- Review specialists and users
- Manage platform content
- Monitor system activities

Main Actions
- Approve specialist profiles
- Manage users
- Control review visibility
- Check logs and announcements

Pain Points
- High management workload
- Need clear system visibility
- Difficult quality control

Customer:
Discovery-oriented, not search-driven

- Core friction: Distrust and uncertainty in every stage;
- Platform's role: reduce uncertainty, not add steps;
- Solution: Certificates and shared reviews.

Specialists
Want System's cooperation

- Availability can be scheduled at convenience;
- Booking requests can show clearly;
- Reputation accumulates without manual effort.

Administrators
Require full control to system transactions

- Specialist approval gates customer exposure
- Anomalies flagged, not silently passed
- Every action logged and traceable

2.1 Architecture Design

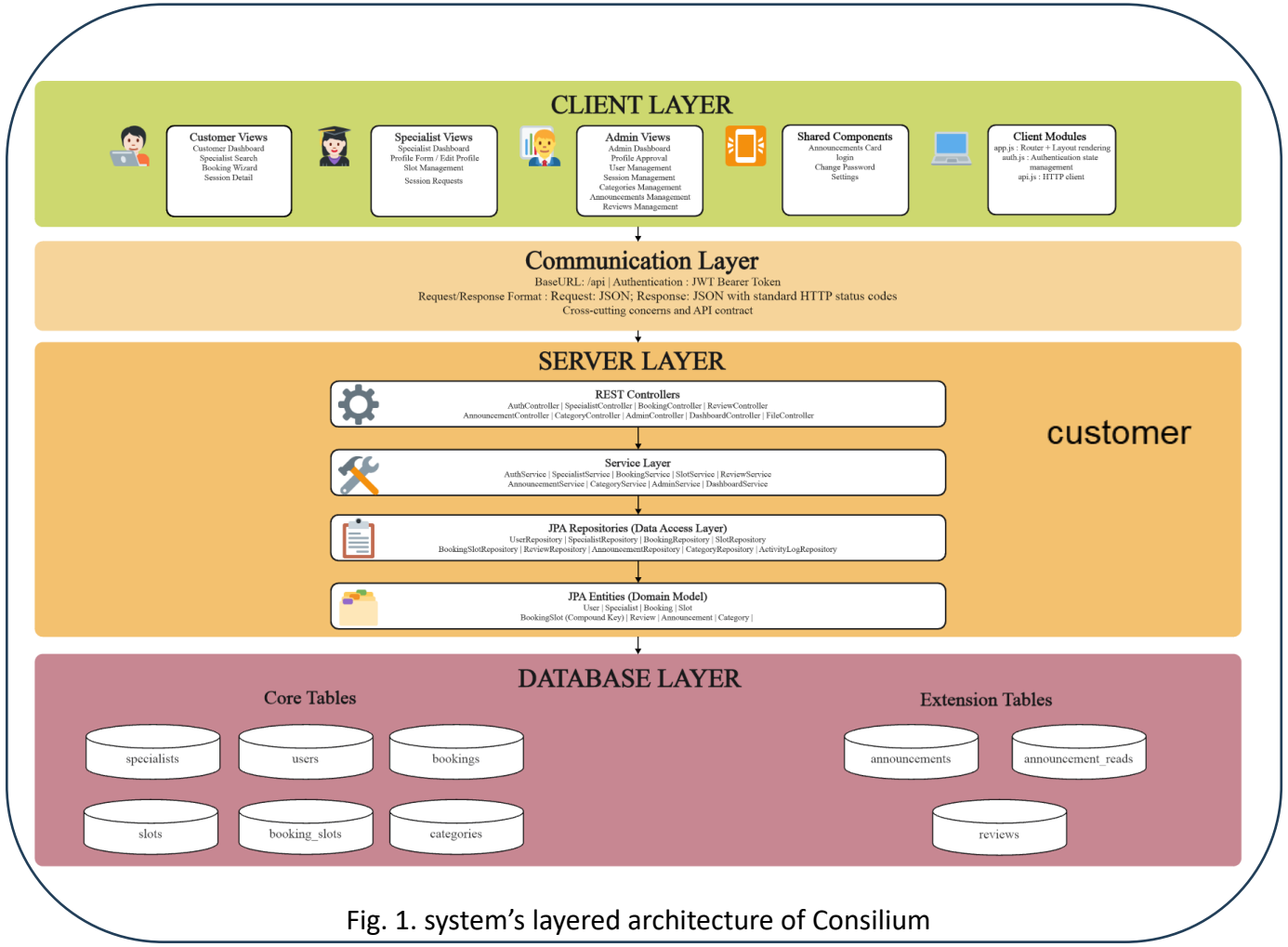


Fig. 1. system's layered architecture of Consilium

Consilium's System Architecture

- Three-tier: SPA frontend, Spring Boot backend, MySQL database
- RESTful API decouples frontend and backend; standard HTTP status codes throughout
- Role-based access control scoped per endpoint: customer, specialist, admin
- Dockerised deployment on an assigned server

2.2 Software Design

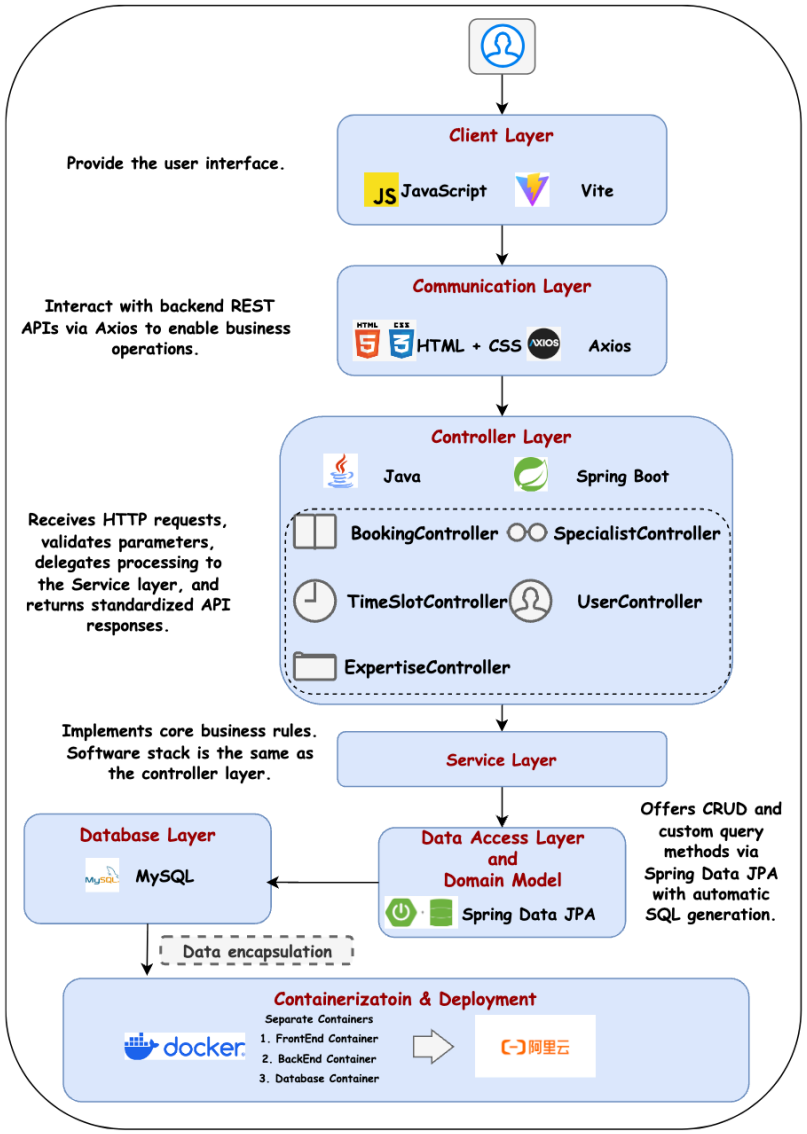


Fig. 1. System architecture and selected techs

Technical Stack

- **Frontend**: Vite, HTML, CSS, JavaScript, Node.js
- **Backend**: Spring Boot 3, Spring Security, JPA/Hibernate, Java 17
- **Database**: MySQL
- **Authentication**: JWT Authentication, Role-based Access Control
- **Deployment**: Docker, Alibaba Cloud
- **Communication**: RESTful API, Bearer Token Requests
- **Version Control**: GitHub
- **Documentation**: Overleaf
- **System Design**: Visual Paradigm

2.2 Software Design

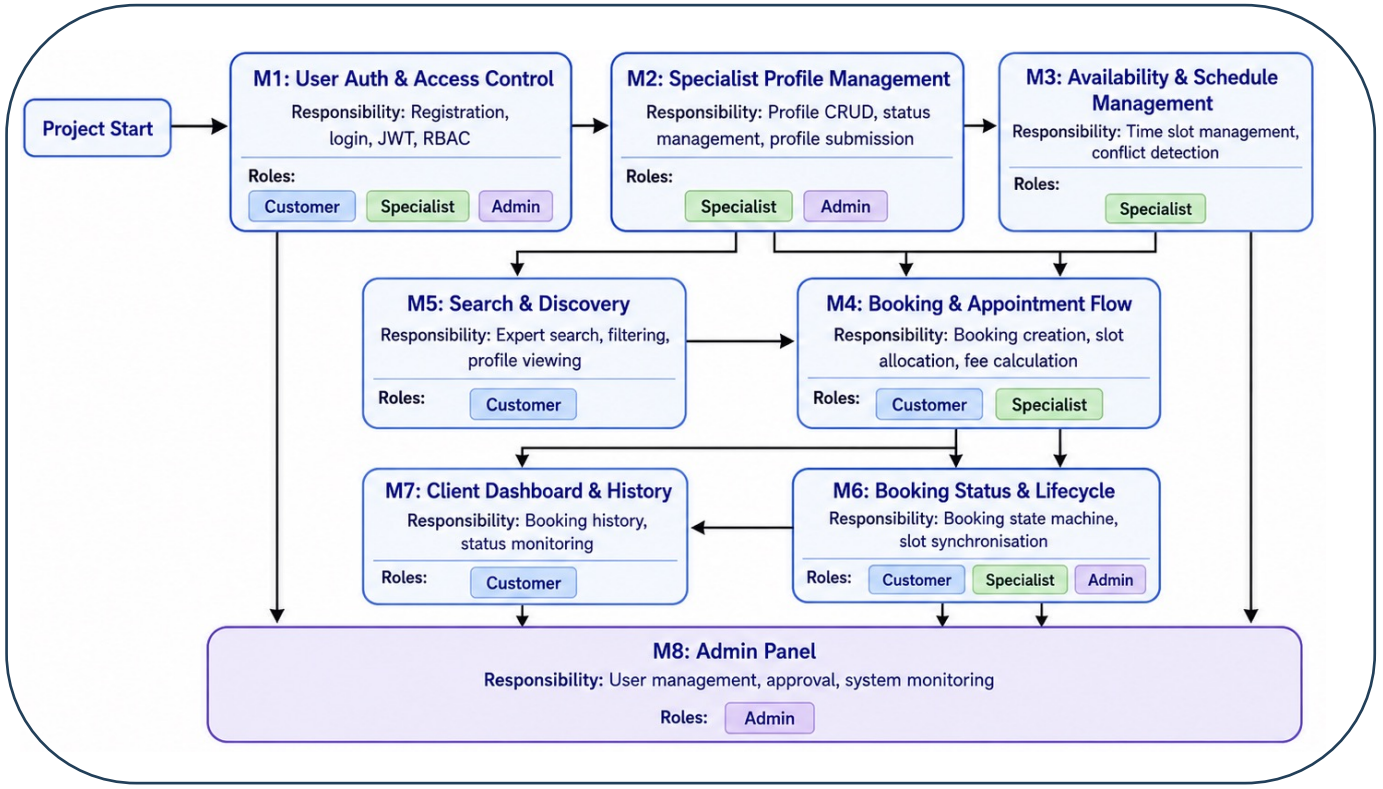


Fig. 1. Core Functional Modules of Consilium's System

8 high-level Modules

- User Auth & Access Control
- Specialist Profile Management
- Availability & Schedule Management
- Booking & Appointment Flow
- Search & Discovery
- Booking Status & Lifecycle
- Customer Dashboard & History
- Admin Panel

Core Module 1: User auth & access control

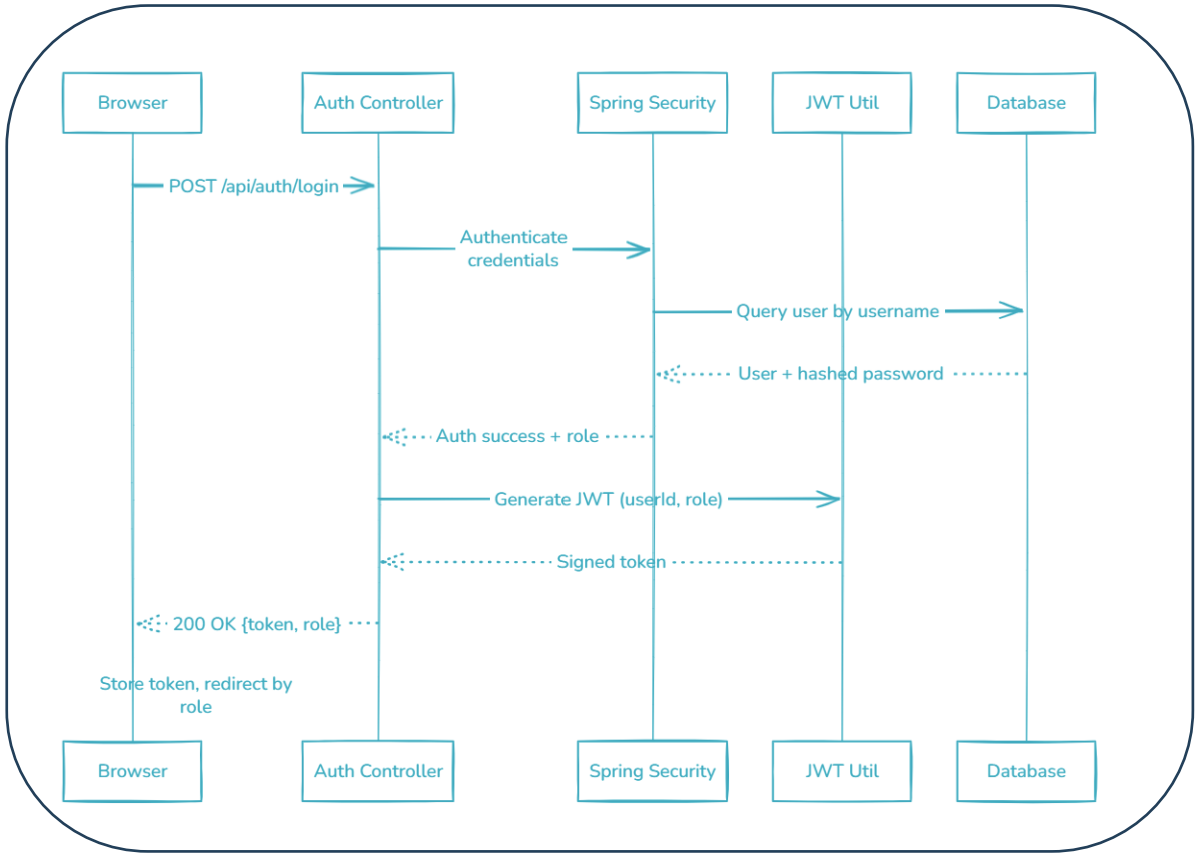


Fig. 1. Login Sequence

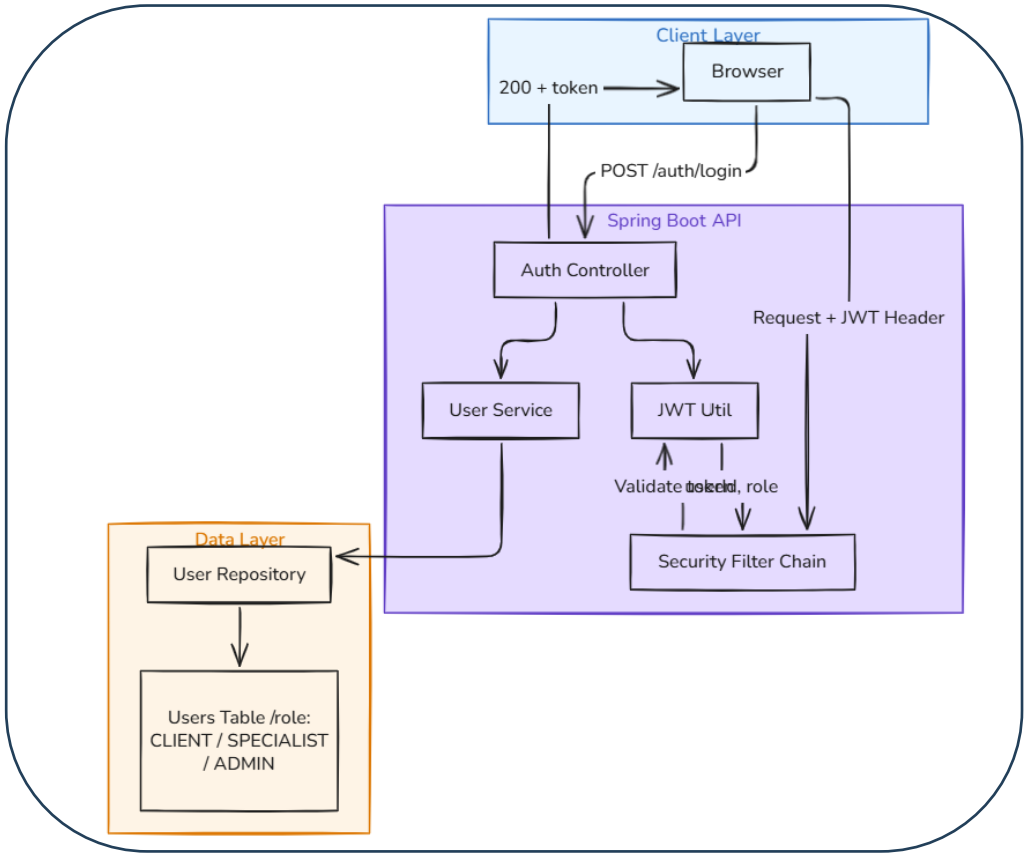


Fig. 2. Auth Component Architecture

Core Module 2: Booking Status Lifecycle

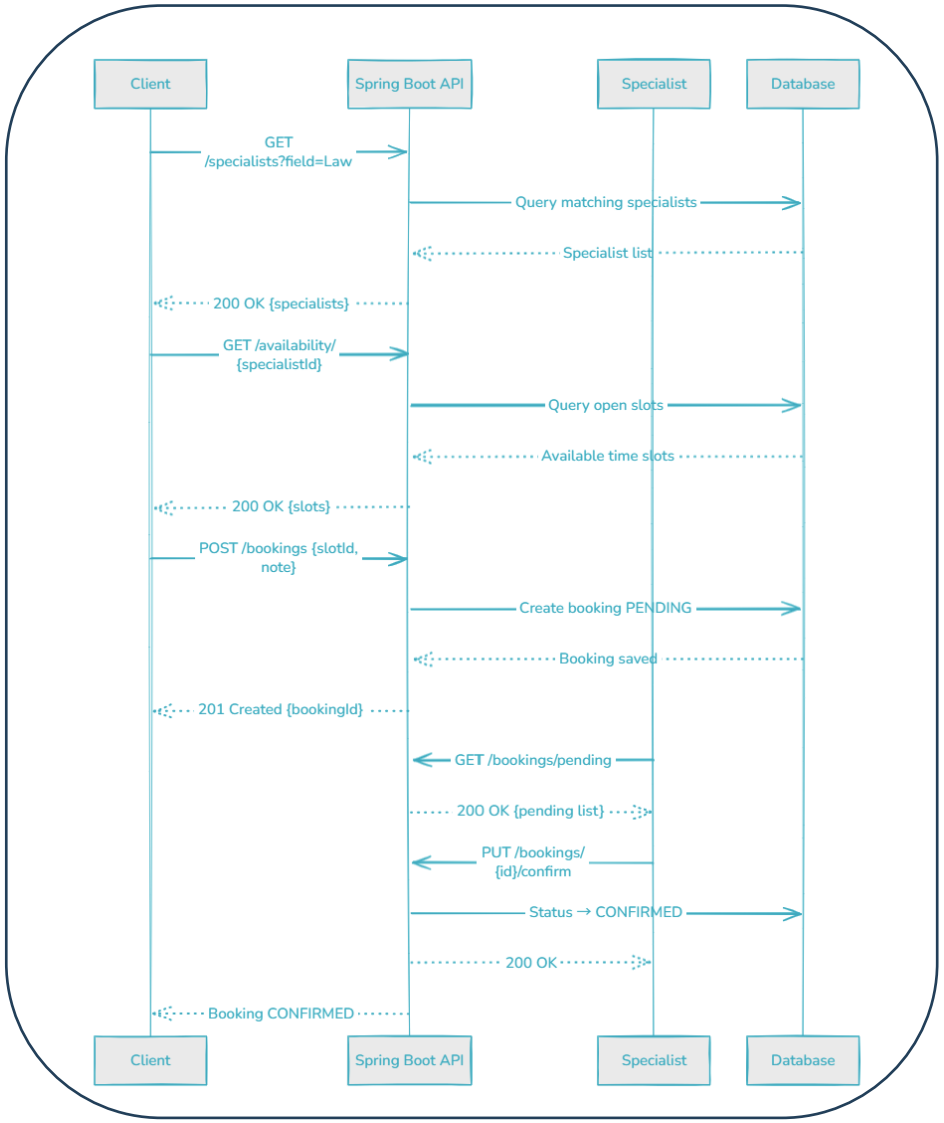


Fig. 1. Booking Flow Sequence

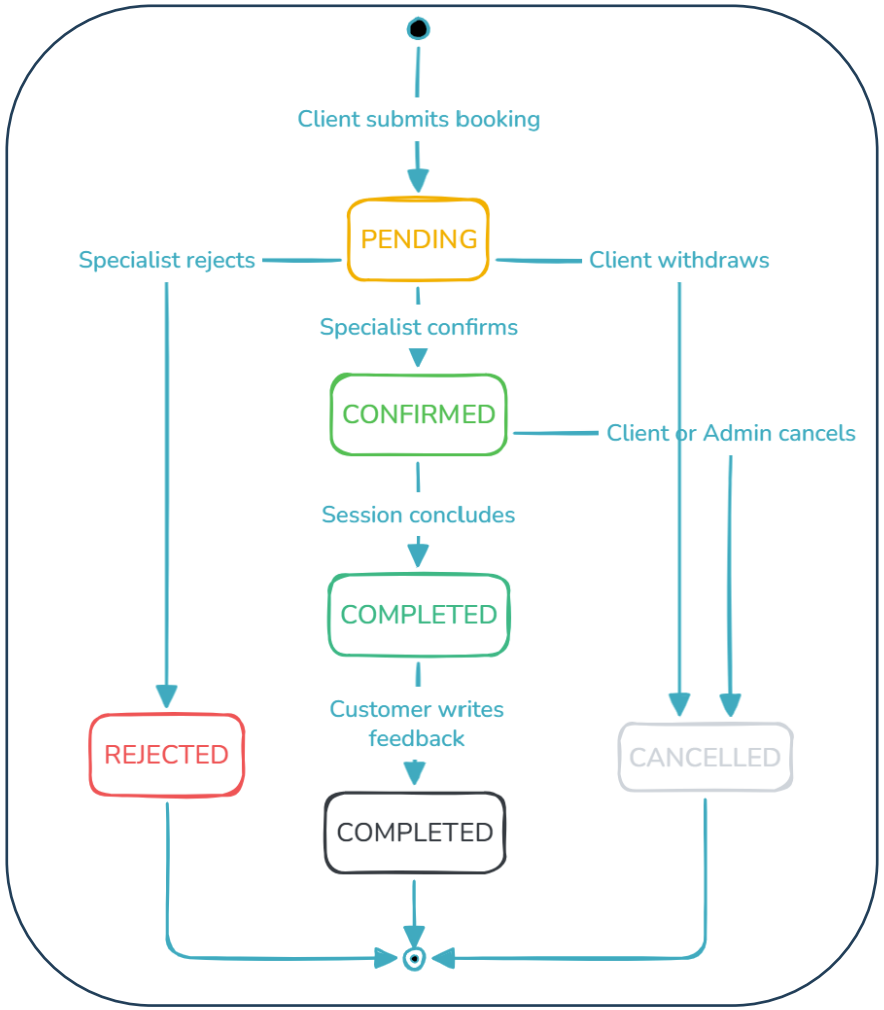


Fig. 2. Booking Status Lifecycle

2.3 Database Design

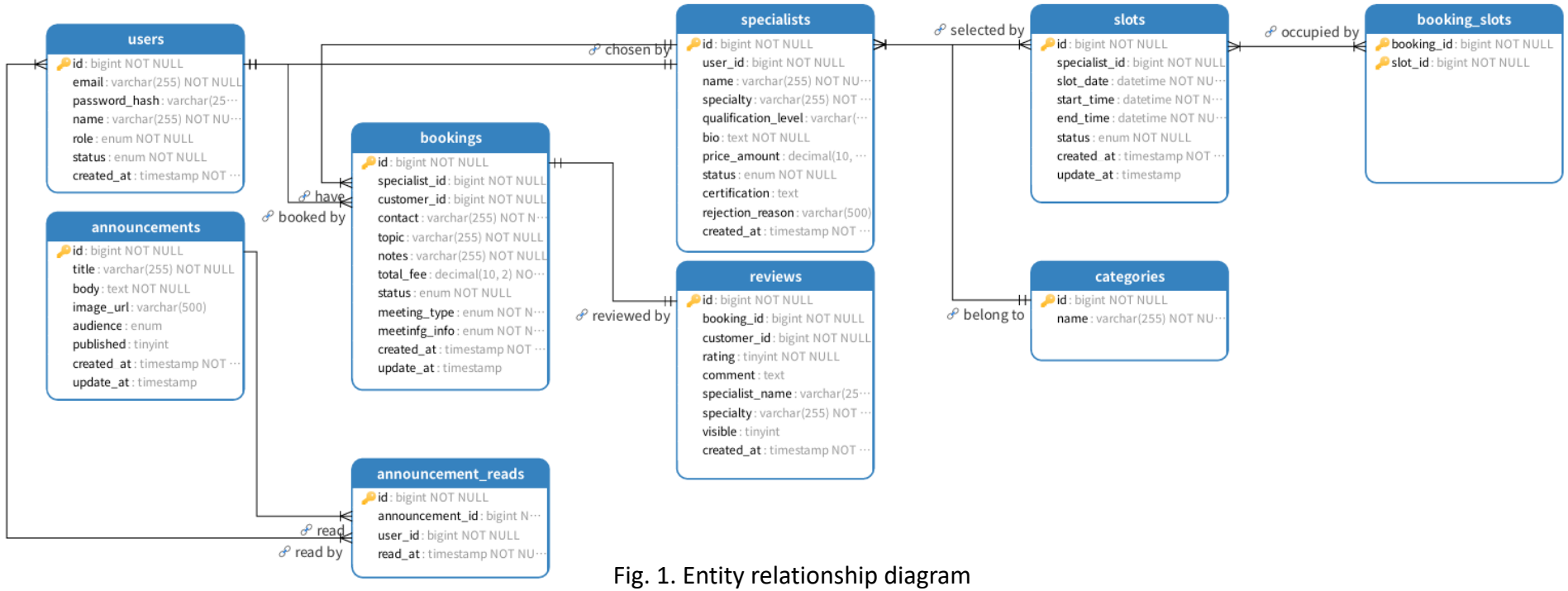


Fig. 1. Entity relationship diagram

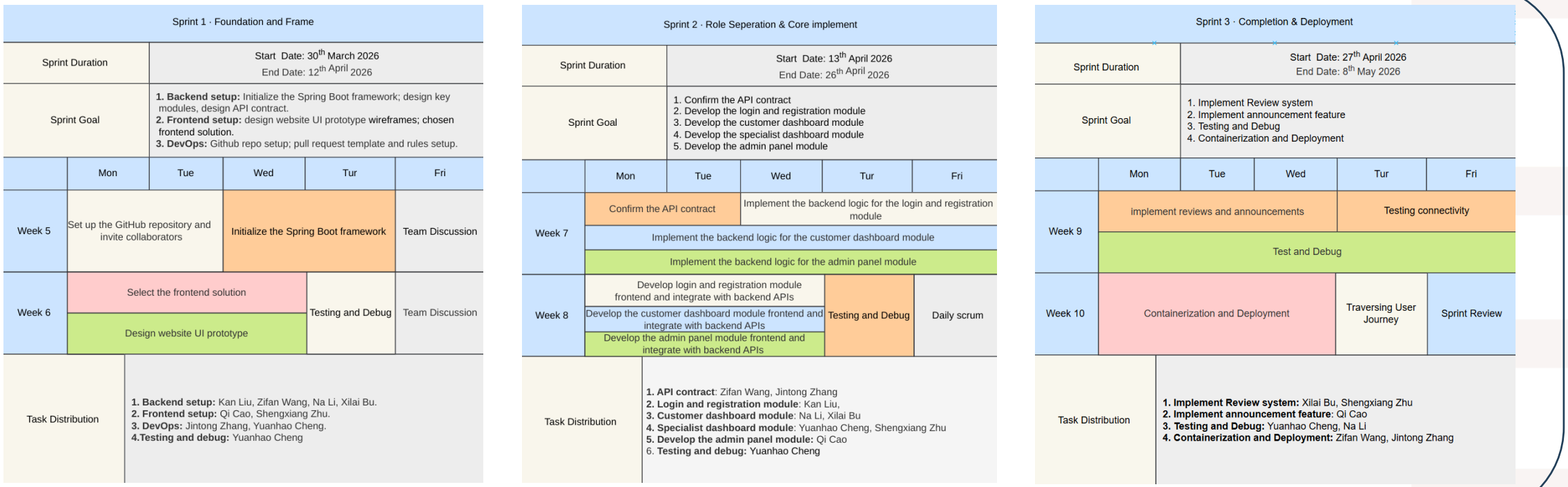
Core Entities:

- users
- Specialists
- slots
- bookings
- booking_slots

Design Highlights

- Clear User–Specialist Relationship
- Structured Booking and Time-slot Management
- Conflict Prevention through Slot Status Control
- Role-based Access and Data Separation
- Scalable Relational Database Design
- Support for Reviews, Logs, and Admin Governance

3. Development process



Sprint 1 · Foundation and Frame

Sprint Duration	Start Date: 30th March 2026 End Date: 12th April 2026
Sprint Goal	**1. Backend setup:** Initialize the Spring Boot framework; design key modules, design API contract. **2. Frontend setup:** design website UI prototype wireframes; chosen frontend solution. **3. DevOps:** Github repo setup; pull request template and rules setup.

	Mon	Tue	Wed	Tur	Fri
Week 5	Set up the GitHub repository and invite collaborators		Initialize the Spring Boot framework		Team Discussion
Week 6	Select the frontend solution Design website UI prototype			Testing and Debug	Team Discussion

Task Distribution	**1. Backend setup:** Kan Liu, Zifan Wang, Na Li, Xilai Bu. **2. Frontend setup:** Qi Cao, Shengxiang Zhu. **3. DevOps:** Jintong Zhang, Yuanhao Cheng. **4.Testing and debug:** Yuanhao Cheng

Sprint 2 · Role Seperation & Core implement

Sprint Duration	Start Date: 13th April 2026 End Date: 26th April 2026
Sprint Goal	1. Confirm the API contract 2. Develop the login and registration module 3. Develop the customer dashboard module 4. Develop the specialist dashboard module 5. Develop the admin panel module

	Mon	Tue	Wed	Tur	Fri
Week 7	Confirm the API contract		Implement the backend logic for the login and registration module		
	Implement the backend logic for the customer dashboard module				
	Implement the backend logic for the admin panel module				
Week 8	Develop login and registration module frontend and integrate with backend APIs Develop the customer dashboard module frontend and integrate with backend APIs Develop the admin panel module frontend and integrate with backend APIs			Testing and Debug	Daily scrum

Task Distribution	**1. API contract**: Zifan Wang, Jintong Zhang **2. Login and registration module**: Kan Liu, **3. Customer dashboard module**: Na Li, Xilai Bu **4. Specialist dashboard module**: Yuanhao Cheng, Shengxiang Zhu **5. Develop the admin panel module:** Qi Cao 6. **Testing and debug:** Yuanhao Cheng

Sprint 3 · Completion & Deployment

Sprint Duration	Start Date: 27th April 2026 End Date: 8th May 2026
Sprint Goal	1. Implement Review system 2. Implement announcement feature 3. Testing and Debug 4. Containerization and Deployment

	Mon	Tue	Wed	Tur	Fri
Week 9	implement reviews and announcements			Testing connectivity	
	Test and Debug				
Week 10	Containerization and Deployment			Traversing User Journey	Sprint Review

Task Distribution	**1. Implement Review system:** Xilai Bu, Shengxiang Zhu **2. Implement announcement feature**: Qi Cao **3. Testing and Debug:** Yuanhao Cheng, Na Li **4. Containerization and Deployment:** Zifan Wang, Jintong Zhang

Fig. 1. Sprint Collaboration details

Collaboration & Tooling

- **GitHub** for version control, code review, and sprint branch management
- **Project board** for backlog tracking and sprint task assignment
- **WeChat** for daily team communication and quick decision-making
- **Shared documentation** maintained throughout all sprints

Development Principles

- **Scrum-driven:** sprints with defined goals, reviews, and retrospectives
- **Incremental:** each sprint delivers a testable, integrated increment
- **Backlog-first:** every feature starts as a user story
- **Definition of Done** enforced across code, UI, and API

4.1 Demonstration

System URL: http://121.41.36.107/

Walkthrough of 3 Core User Flows

- **Customer Flow — Search, discover, and submit a booking request**
- **Specialist Flow — Respond to requests and manage profile under governance**
- **Admin Flow — Approve specialists, oversee users, sessions, and announcements**

4.2 Technical Follow-up Highlights

Core Module Interactions

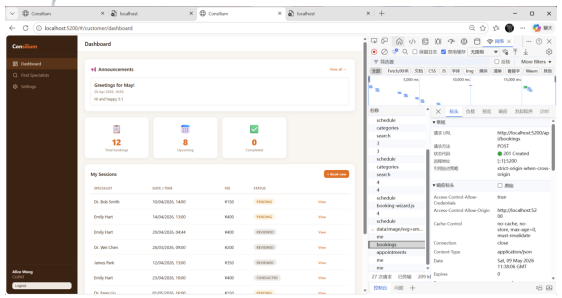


Fig. 1.1 Customer; Booking Creaion: POST /api/bookings → 201 Created

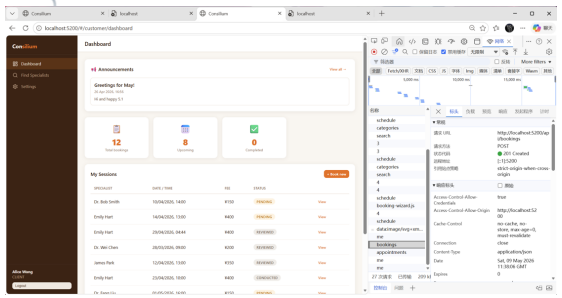


Fig. 1.2 Specialist; Confirm Booking: PATCH /api/bookings/{id}/confirm → 200 OK

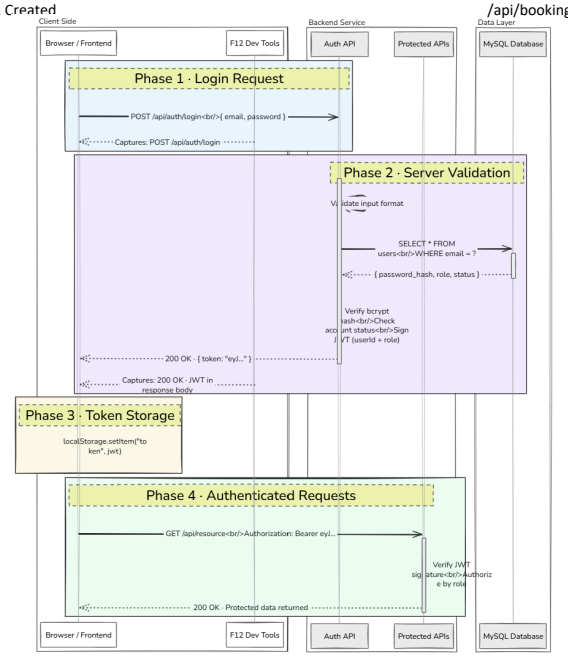


Fig. 1.3 Sequence Diagram demonstrating interactions of frontend, backend and DB

Auth Key Validation

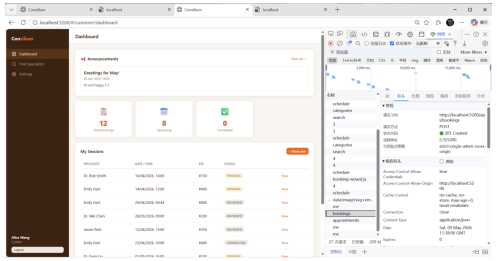
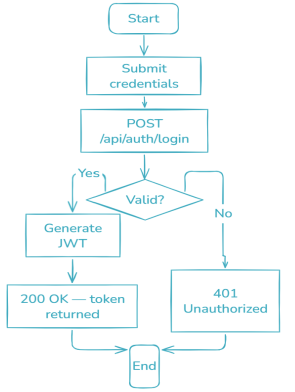


Fig. 1.1 User Authentication JWT token, POST /api/auth/login → 200 OK



Slot Conflict Prevention

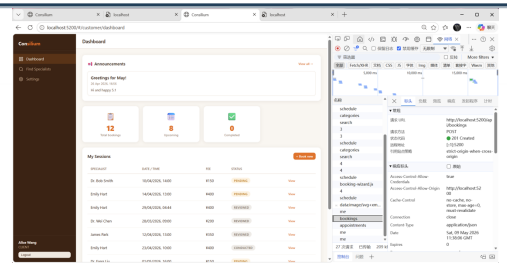
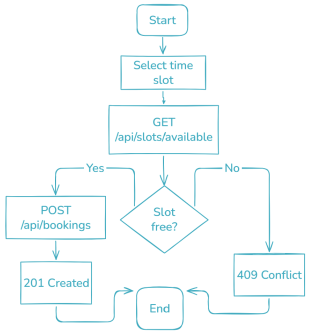


Fig. 1.1 User Authentication JWT token, POST /api/auth/login → 200 OK



Access Control Enforcement

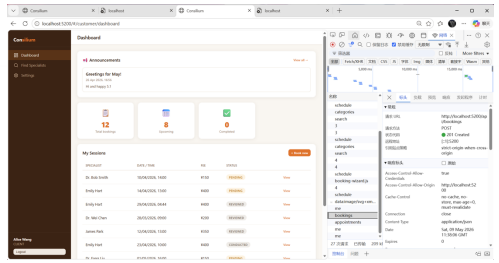


Fig. 1.1 User Authentication JWT token, POST /api/auth/login → 200 OK

4.3 Testing Strategy & Documentation

Testing Focus

- Core booking workflow
- Authentication and role-based access control
- Time-slot availability and conflict prevention
- Backend API correctness
- Customer, Specialist, and Admin acceptance scenarios

Testing Types

- Unit Testing: verify service-layer business logic
- Integration Testing: verify REST APIs and database effects
- Acceptance Testing: verify end-to-end user workflows

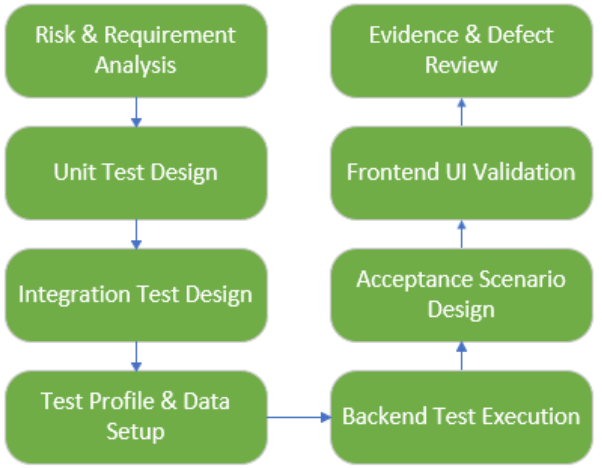


Fig. 1. Global Testing Architecture

Fig. 2. Integration testing example: customer booking flow

4.3 Testing Strategy & Evidence

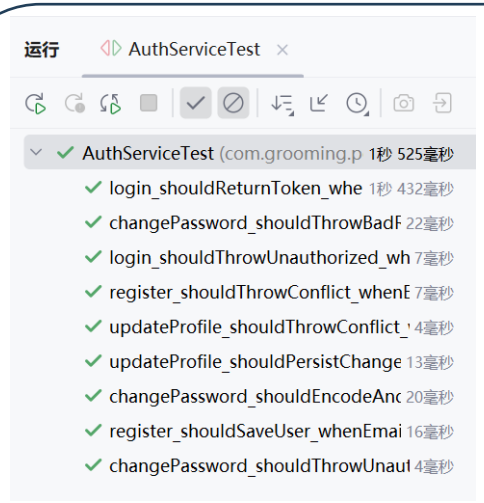


Fig. 1. AuthServiceTest All Passed

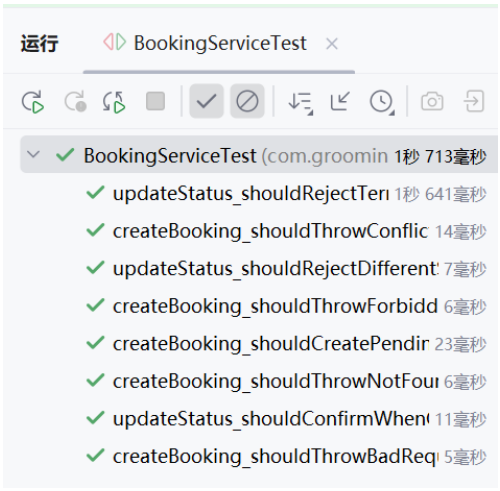


Fig. 2. BookingServiceTest All Passed

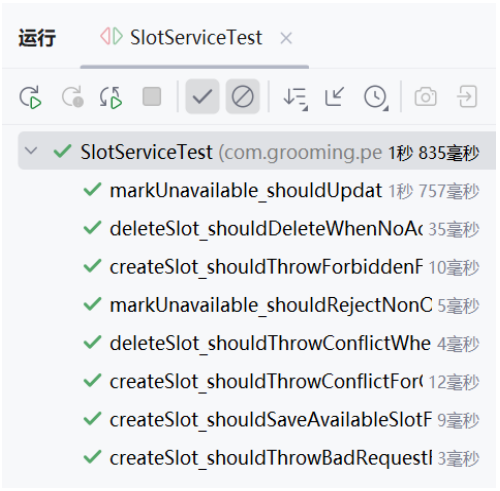


Fig. 3. SlotServiceTest All Passed

Unit Testing

- JUnit 5 and Mockito
- Service-layer logic verified independently
- Covered authentication, booking, slot, specialist, and admin services

Integration Testing

- Spring Boot Test and MockMvc
- Verified API responses and database updates
- Checked role permissions and ownership rules

Acceptance and Boundary Testing

- Customer booking workflow verified
- Specialist confirmation workflow verified
- Admin management workflow verified
- Invalid input, unauthenticated access, duplicate booking, and unauthorized access

5 Conclusion

Achievements:

- Completed role-based system for Customer, Specialist, and Admin
- Implemented registration, login, JWT authentication, and RBAC
- Supported specialist search, profile management, and availability management
- Implemented booking creation, confirmation, cancellation, conducted, and reviewed states
- Provided admin governance for users, specialists, reviews, categories, announcements, and logs
- Improved the process to be more efficient, transparent, and manageable

Limitations

- Limited notification support
- No payment and refund management
- Limited real-time communication
- Basic mobile responsiveness
- Limited large-scale deployment testing

Future Work

- Add email or system notifications
- Add online payment and refund features
- Integrate chat or video meeting support
- Improve responsive UI design
- Add admin analytics and statistical reports
- Conduct more production-level testing

THANK YOU

Xi'an Jiaotong-Liverpool University

西交利物浦大学